UNIVERSITY of HOUSTON
CULLEN COLLEGE of ENGINEERING

Job Recommendation System

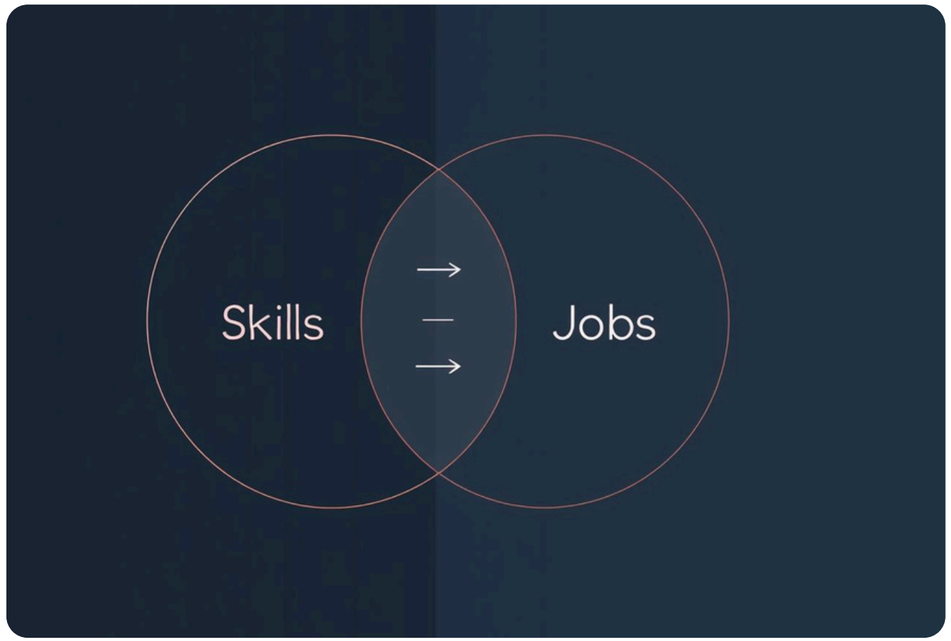
Presented By
Yatrikkumar Shah
Shail Shah
Abrar Hamim Masih
Anish Khanal

Problem Statement

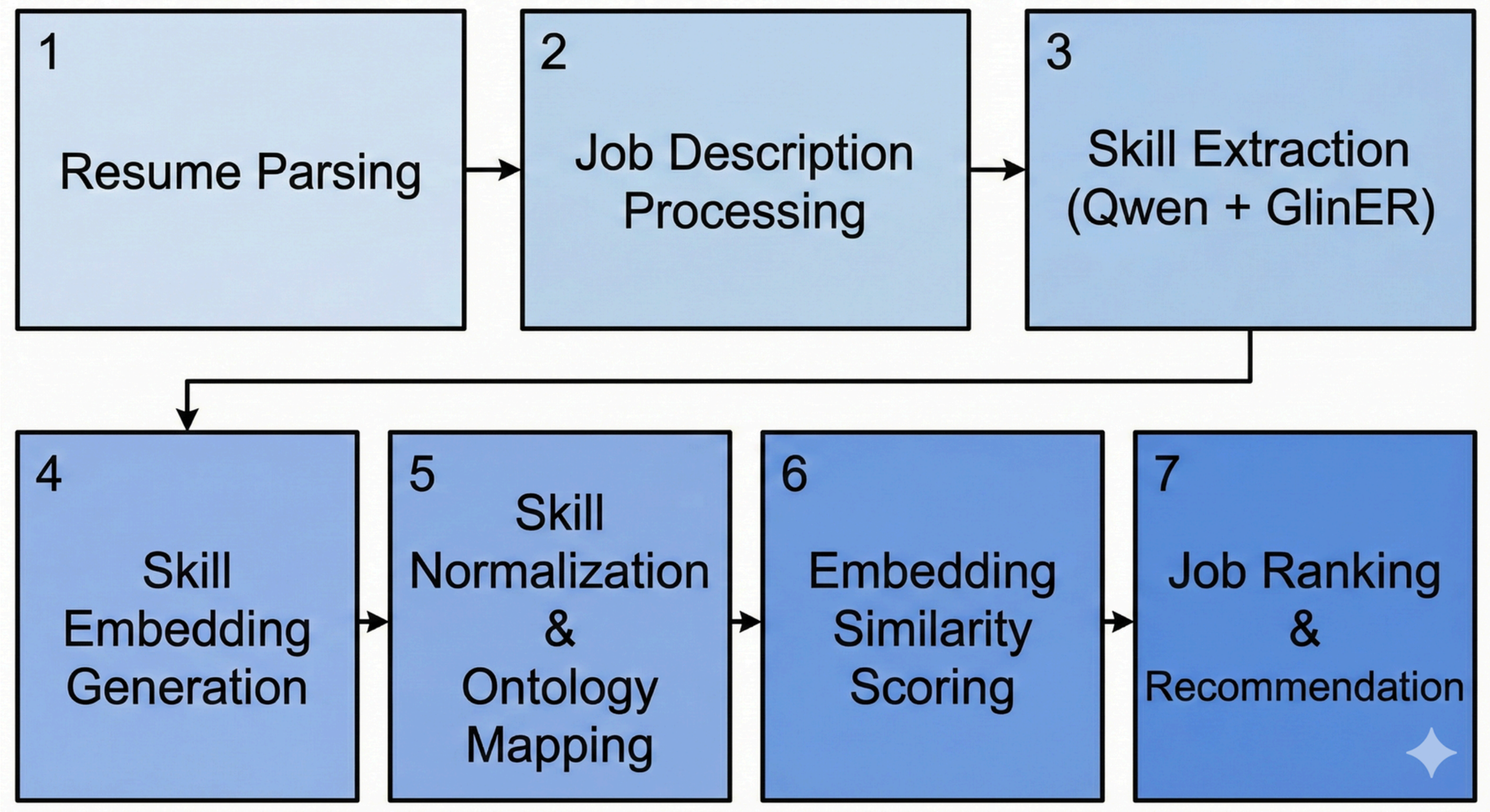
- Job seekers often struggle to identify roles that truly align with their skills.
- Recruiters receive large volumes of resumes and struggle to filter candidates efficiently.
- Traditional keyword-based matching fails to capture semantic meaning, skill variations, and context, for example, LinkedIn standard search
- There is no standardized structure in resumes or job descriptions, making matching inconsistent.

Proposed Solution

- Extract Skills from Resume & JD Using NLP + LLMs
- Convert Skills into Semantic Embeddings
- Compute Resume - Job Similarity
  - Skills - Requirements similarity
  - Experience - Responsibilities similarity
- More Filtering and Job Searching features
- Rank & Recommend Jobs



System Architecture



Resume Parsing Flow

- Document Ingestion
  - Imports resumes in PDF and extracts the text
- Preprocessing & Cleaning
  - Normalize whitespace and punctuation, Lowercase transformation, removes non-UTF characters
- Section Identification: Skills - Experience
  - LLM-Based Extraction
    - Understands context in sentences - thinks - reasons - extracts
    - Extracts explicit & implicit skills
  - NER-Based Extraction
    - Supervised Named Entity Recognition
    - Detects domain-specific skill entities with high precision
    - Detects common resume sections like: Experience, Education, Skills, Projects, Certifications.

Job Description Parsing Flow

- JD Ingestion
  - Accepts job descriptions from LinkedIn, Indeed, or manual paste
- Preprocessing & Cleaning
- Section Identification: Requirements and Responsiblities
  - LLM-Based Extraction
    - More accurate
    - Time Taking - depends on the system
    - Works on the given instruction
  - GlinER NER-Based Extraction
    - Faster than core LLM-Based
    - Works on the Embeddings-based token-level classification
    - Hybrid NER and contextual understanding
    - Limited to the explicit skills

Skill Normalization & Ontology Mapping

- The extracted skill list contains many duplicates, variants, shorthand, or synonyms
- “Deep learning”, “DL”, “Neural nets” → must resolve to one standard term
- “Pandas”, “pandas library”, “Python dataframes” → group under Data Handling: Pandas
- We align all extracted skills to a canonical skill dictionary (skill-ontology)
- Matching variations are resolved using embedding similarity metrics.

Benefits of normalization:

- Fixes naming inconsistencies
- Merges synonyms to one bucket
- Removes duplicated skill entries
- Enables high-quality semantic job matching

Encoding (Vector Representations)

- **Why needed?**
  - To compare resumes and job descriptions mathematically, we must convert text-based skills into numeric vectors.
  -
- **A. TF–IDF Encoding (Traditional):**
  - Represents text as word-frequency vectors
  - Sparse + high-dimensional
  - Does not capture meaning or synonyms

- **B. Transformer Embeddings (Our Approach)**
  - Uses pretrained models (Sentence Transformers)
  - dense semantic vector for JD and Resume
  - Captures the meaning and context of the whole document

Ranking & Recommendation

- Compute similarity across all job postings.
- Sort jobs by highest similarity score → top-N selected.
- Recommendation result includes:
  - Matching Skills
  - Missing Skills (gap analysis)
  - Similarity Confidence Score
  - Profile Match Strength

This makes results actionable for job seekers and screening efficient for recruiters.

Models Used in Our System

- **Qwen (LLM)**
  - Extracts skills, responsibilities, and experience from Resume & JD
  - Understands context and implicit skills
  - strong semantic extraction
- **GlinER (NER Model)**
  - Detects explicit skills and technical entities
  - Token-level precision, low hallucination
- **Sentence Transformer (Embedding Model)**
  - Converts entire Resume & JD into dense semantic vectors
  - Captures meaning, role similarity, and context
  - Enables cosine similarity for matching

Why Unsupervised

- No Labeled Dataset Exists
  - No dataset with “Resume - Job → Good / Bad match” available publicly
  - Creating labels manually is expensive, time-consuming, and subjective
  - Every domain would require separate labeled datasets
  - Supervised learning is impractical due to lack of reliable ground-truth labels.

- LLMs & Embedding Models Already Encode Knowledge
  - Zero-shot semantic similarity works extremely well for this problem

- Similarity-Based Matching Is Naturally Unsupervised

- Easy to maintain and update
  - Unsupervised models can incorporate new skills without retraining

Future Scope

Fine-tune transformer/LLM for skill extraction domain classification

Add reranking using Qwen-Large or LLaMA to improve semantic relevance

Introduce user feedback loop so system learns from:

- jobs user applied to
- jobs user liked
- jobs user skipped
- Add interactive UI dashboard (Streamlit/React)

Include weighted scoring & business filters:

- salary
- location
- experience level
- job category
- recruiter/company rating

Conclusion

We replaced inconsistent keyword search with a semantic + ontology-based matching system

System recommends jobs based on:

- Skills
- Experience
- Context similarity
- Ranking confidence
- This improves efficiency for both job seekers and recruiters.

Thank you!